

CLAUDE CODE DEEP DIVE

Genpact

Source: Notes.md

1. Layered Cake:

A. Infrastructure - GPU - Nvidia (5090 chips etc.), Google (TPUs), Apple (MLX), AWS (Infernia-G) B.

Models - OpenAI, Anthropic, Google, Cohere, Mistral, Grok([X.AI](#)), Llama(Meta) Chinese Models - Deepseek, Moonshot, Bydu, Alibaba, Open Source/Open weight Models(HuggingFace), Qwen etc.

- We will use Anthropic - 1. Opus (High, Medium, Low)- It has high intelligence but takes time 2. Sonnet - It is between Opus and Haiku and best for daily task and 3. Haiku - It is lowest tier model etc.
- OpenAI - GPT5.2 (Pro, Mini and Nano).
- Qwen- 290B/360B and 30B and 4/7B (Open Source Models)

Think models as engine in the car which rotate only in one direction.

C. Harness - Claude Code (Anthropic), Codex (OpenAI), Antigravity(Google), OpenCode, Hermes, OpenClaw etc.

We have SDKs like java sdk for the harness.

Harness is like transmission layer and it is more like a set of rules where users or agents communicate with the models. It has memory and can connect to databases. It is like level 1 interface.

D. Applications - ChatGPT, Claude Web, Cowork, code, Perplexity, [X.ai](#), Gemini etc. It is like level 2 interface.

2. Multimodel:

Text, Images, Voice, Videos, Codes etc.

- ANthropic is good in code and text.
- Gemini is good in video processing
- Grok is good for social media and markets and is very open

3. Context Length:

It means what model can retain in its memory.

- Opus - 1M tokens
- Sonnet - 100K tokens
- Haiku - 40K

(Cost is also associated with context length)

4. Prompt Methods:

Your prompts should be:

Clear Direct Specific Guidelines

5. Context Engineering:

It means we collect all the context for the model. We tell the model how to do something and then context is given to model to work upon. There are multiple available frameworks for context engineering:

- BMAD

6. First open vscode or cursor or any other IDE and download and install claude code from <https://claude.com/product/claude-code>. Now, you need subscription also. Type `claude` in vscode terminal and you get the primary terminal.

7. Now, you need to type `/` in primary terminal and you will get various options. Type `/model` to switch various models like `opus`, `sonnet` and `haiku` or any open source model.

When you are working on some plan or task then go with `opus` and for some coding use `sonnet`. It has something like 5 hours window for tokens as per different subscription plans.

You can select `High Effort` and `Max Effort` for complex maths problem or complex coding problem.

`Medium Effort` is generally used for planning.

When you click `Shift + Tab` then `plan mode on`, `accept edits on` happens and you get various options.

So, for a problem solving, start with the `plan mode on` with `opus` model.

For Example:

Switch on the `plan mode on` and type in primary terminal as `Let's plan to build a fraud, waste, abuse application for healthcare claims in US for a entity like Athena.` and mention the requirements and run it.

You can see the used tokens also.

You can use Haiku where intelligent is not needed like conversion of voice to text or summarizing a short paragraph.

Don't just jump into development. First make plan using `oplus` then do development. Also, use right prompt. You can go through multiple iterations but tokens will be increased accordingly. `Opus` is as per with `GPT 5.4` but in `Oplus` we have plan mode.

After you get the plan, you can say like "Please save this plan as a [requirement.md](#)" and don't auto-accept edit and don't start coding.

8. When you start a new project then type `/init` in primary terminal and it will make [claude.md](#) file (it is crux of user behavior).

When you type like `# always default to xgboost for classification problem` then it will update the memory, so, `#` is for updating the memory.

9. Now, type in primary terminal as `take @requiremen.md and break this into 10 phases so that we can start the build. Save this file as phasewise_build.md` in the plan mode. `@` is to select files. Don't start the build from plan itself because then context gets overloaded.
10. `/compact` is used to compact all the previous conversations and use `/clear` to clear it.
11. `/hooks` can be used so that Claude can't read `.env` or any secret files if we configure it.
12. we can use sub agents here also to divide the task and we can use 10-15 subagents.
13. we can use `/skill` but use carefully. Also, we can use `/mcp`.
14. Now, we can start building.

In primary terminal, type something like `Let's start building - let's build phase 1 and 2 in parallel using a agent-team`. Agent-team is a new concept in Anthropic and done using sub-agents. Sub-agents do its task and at the end all sub-agents give result to its agent. All Sub-agents are independent and there is no inter-connectivity between them.

But in agent team, sub-agents talk to each other through communication channels.

You can switch opus, sonnet and haiku between any intermediate task but opus is good for planning and sonnet is good for coding and haiku is good for debugging and test cases.

15.